Poly de cours – Vision par ordinateur, réseaux convolutifs et explicabilité

Niveau L2–L3 / CPGE avec ouverture vers la pratique ML

Document pédagogique orienté mathématiques + pratique machine learning

Mars 2026

Objectif : comprendre rigoureusement, mais progressivement, comment on passe d'une image numérique à une prédiction par réseau de neurones convolutif, puis à une explication visuelle de type Grad-CAM.

Table des matières

1 Introduction générale **2**

2 Représentation mathématique d'une image **2**

2.1 Image en niveaux de gris 2

2.2 Image couleur 2

2.3 Image comme vecteur de grande dimension 2

3 Prétraitement des images **3**

3.1 Normalisation 3

3.2 Standardisation 3

3.3 Redimensionnement 3

3.4 Augmentation de données 3

4 Convolution discrète : définition et intuition **4**

4.1 Définition en 1D 4

4.2 Définition en 2D 4

4.3 Interprétation locale 4

4.4 Exemple : filtre détecteur de contour vertical 4

4.5 Exemple : filtre de lissage 4

5 Convolution multi-canaux **5**

6 Pourquoi la convolution est plus intelligente qu'une couche dense **5**

7 Fonctions d'activation **5**

7.1 Pourquoi une non-linéarité ? 5

7.2 ReLU . . . 6
7.3 Sigmoïde . . . 6
7.4 Tanh . . . 6

8 Pooling et réduction de dimension **6**
8.1 Max pooling . . . 6
8.2 Average pooling . . . 6
8.3 Pourquoi cela aide ? . . . 6

9 Champ récepteur **6**

10 Global Average Pooling **7**

11 Couches denses et classifieur final **7**

12 Softmax et interprétation probabiliste **7**

13 Fonctions de perte **7**
13.1 Cross-entropy binaire . . . 7
13.2 Cross-entropy multi-classe . . . 8

14 Descente de gradient **8**
14.1 Mini-batch SGD . . . 8

15 Backpropagation : principe général **8**

16 Gradient d'une couche dense **8**

17 Gradient d'une convolution : idée générale **9**

18 Architecture d'un CNN simple pour radiographies **9**
18.1 Paramètres de la première couche . . . 9
18.2 Deuxième couche convolutive . . . 9
18.3 Troisième couche convolutive . . . 9

19 Régularisation **9**
19.1 Weight decay . . . 9
19.2 Dropout . . . 10
19.3 Early stopping . . . 10

20 Optimiseur Adam **10**

21 Mesures de performance **10**
21.1 Accuracy . . . 10

21.2 Précision 10
21.3 Rappel 10
21.4 F1-score 11

22 Pourquoi l'explicabilité est nécessaire **11**

23 Principe de Grad-CAM **11**

24 Pourquoi la dernière couche convolutionnelle ? **11**

25 Exemple jouet de Grad-CAM **11**

26 Limites de Grad-CAM **12**

27 Exemple pratique en PyTorch **12**

28 Exemple pratique en TensorFlow / Keras **12**

29 Lien avec le projet MedVision **13**

30 Exercices proposés **13**

31 Conclusion générale **14**

A Bibliographie minimale conseillée **14**

1 Introduction générale

La vision par ordinateur consiste à construire des algorithmes capables d'extraire de l'information à partir d'images. Dans un contexte de classification, on cherche à approximer une fonction

$$f_\theta : \mathbb{R}^{H \times W \times C} \to \mathbb{R}^K,$$

où :

- H est la hauteur de l'image,
- W sa largeur,
- C le nombre de canaux (par exemple $C = 3$ en RGB),
- K le nombre de classes.

Dans ce poly, nous voulons répondre à quatre questions :

1) Comment représenter mathématiquement une image ?
2) Pourquoi la convolution est-elle l'opération centrale des CNN ?
3) Comment apprend-on les paramètres d'un réseau ?
4) Comment expliquer la décision du réseau, notamment en imagerie médicale ?

2 Représentation mathématique d'une image

2.1 Image en niveaux de gris

Une image en niveaux de gris peut être vue comme une fonction discrète

$$I : \{1, \dots, H\} \times \{1, \dots, W\} \to \{0, \dots, 255\}.$$

Autrement dit, à chaque paire (i, j) on associe une intensité lumineuse. Il est commode de la représenter sous forme de matrice

$$I \in \mathbb{R}^{H \times W}.$$

Si l'image est normalisée, on travaille souvent avec

$$I_{\text{norm}}(i, j) = \frac{I(i, j)}{255} \in [0, 1].$$

2.2 Image couleur

Une image couleur RGB possède trois canaux : rouge, vert, bleu. Mathématiquement :

$$I \in \mathbb{R}^{H \times W \times 3}.$$

On note souvent $I_{i,j,c}$ l'intensité au pixel (i, j) dans le canal $c \in \{1, 2, 3\}$.

2.3 Image comme vecteur de grande dimension

Une image peut aussi être « dépliée » en un vecteur de dimension HWC :

$$\mathrm{vec}(I) \in \mathbb{R}^{HWC}.$$

Par exemple, une image $224 \times 224 \times 3$ correspond à

$$224 \times 224 \times 3 = 150\,528$$

composantes réelles.

Interprétation importante. Une image est donc un point dans un espace vectoriel de très grande dimension. La vision par ordinateur consiste à apprendre des fonctions utiles sur cet espace.

3 Prétraitement des images

Avant l'apprentissage, on applique souvent plusieurs transformations.

3.1 Normalisation

La normalisation la plus simple consiste à diviser par 255 :

$$X' = \frac{X}{255}.$$

Pourquoi ? Parce que l'optimisation est plus stable quand les variables sont de même ordre de grandeur.

3.2 Standardisation

On peut aussi recentrer-réduire :

$$X'' = \frac{X - \mu}{\sigma},$$

avec μ moyenne empirique et σ écart-type empirique.

3.3 Redimensionnement

Pour pouvoir faire entrer toutes les images dans le même modèle, on les redimensionne à une taille fixe, par exemple 224×224.

3.4 Augmentation de données

L'idée est de fabriquer de nouvelles images plausibles à partir des images existantes : rotations, translations légères, symétries, variations de luminosité, zoom léger. Mathématiquement, on applique une transformation aléatoire

$$T : \mathcal{X} \to \mathcal{X}$$

où $\mathcal{X}$ désigne l'espace des images.

4 Convolution discrète : définition et intuition

4.1 Définition en 1D

En analyse, tu connais la convolution continue :

$$(f * g)(x) = \int_{\mathbb{R}} f(t)g(x-t)\,dt.$$

Dans le cas discret 1D, on a :

$$(f * g)[n] = \sum_k f[k]g[n-k].$$

4.2 Définition en 2D

Pour une image I et un noyau K de taille $k \times k$, la convolution discrète 2D s'écrit

$$(I * K)(i,j) = \sum_{u=0}^{k-1}\sum_{v=0}^{k-1} K(u,v)I(i+u, j+v).$$

Selon les conventions, on peut écrire $I(i-u, j-v)$ ou $I(i+u, j+v)$. En pratique, en deep learning, on emploie souvent l'opération de *corrélation croisée*, très proche de la convolution.

4.3 Interprétation locale

La convolution prend un petit voisinage de l'image et calcule un produit scalaire avec le noyau. C'est donc une mesure de ressemblance entre un motif local et le filtre.

4.4 Exemple : filtre détecteur de contour vertical

$$K = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}.$$

Si la région locale présente une différence forte entre gauche et droite, le résultat est grand en valeur absolue. Ce filtre détecte donc les contours verticaux.

4.5 Exemple : filtre de lissage

$$K = \frac{1}{9}\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}.$$

Ce filtre calcule une moyenne locale : il lisse l'image.

5 Convolution multi-canaux

Dans les CNN, on ne travaille pas seulement sur des images en niveaux de gris. L'entrée d'une couche convolutive est souvent un tenseur

$$X \in \mathbb{R}^{H \times W \times C_{\text{in}}}.$$

Un filtre a alors la forme

$$W \in \mathbb{R}^{k \times k \times C_{\text{in}}}.$$

Si on utilise C_{out} filtres, la sortie a la forme

$$Y \in \mathbb{R}^{H' \times W' \times C_{\text{out}}}.$$

Chaque composante s'écrit

$$Y_{i,j,f} = \sum_{c=1}^{C_{\text{in}}} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} W_{u,v,c,f} X_{i+u,j+v,c} + b_f.$$

Idée essentielle : chaque filtre combine tous les canaux d'entrée mais produit une seule carte de sortie.

6 Pourquoi la convolution est plus intelligente qu'une couche dense

Supposons une image $224 \times 224 \times 3$. Une couche dense vers 100 neurones nécessiterait

$$224 \times 224 \times 3 \times 100 = 15\,052\,800$$

poids, ce qui est énorme.

Avec une convolution de 16 filtres 3×3, on a

$$(3 \times 3 \times 3 + 1) \times 16 = 448$$

paramètres seulement.

Conclusion. La convolution exploite deux hypothèses très fortes : la localité spatiale et le partage des poids.

7 Fonctions d'activation

7.1 Pourquoi une non-linéarité ?

Si l'on empile uniquement des transformations linéaires, la composition reste linéaire. Donc un réseau profond sans non-linéarité n'est pas plus expressif qu'une seule matrice.

7.2 ReLU

La fonction ReLU est définie par

$$\mathrm{ReLU}(x) = \max(0, x).$$

Ses avantages : calcul très simple, gradients non nuls pour $x > 0$, activations clairsemées.

7.3 Sigmoïde

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Elle est historiquement importante mais souffre souvent de saturation.

7.4 Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

Elle est centrée mais peut aussi saturer.

8 Pooling et réduction de dimension

8.1 Max pooling

Sur une fenêtre 2×2, le max pooling calcule

$$y = \max(x_1, x_2, x_3, x_4).$$

Cela réduit la taille spatiale et conserve l'activation la plus forte.

8.2 Average pooling

On peut aussi prendre la moyenne :

$$y = \frac{x_1 + x_2 + x_3 + x_4}{4}.$$

8.3 Pourquoi cela aide ?

Le pooling réduit le coût calculatoire, introduit une certaine invariance aux petites translations et augmente la taille du champ récepteur effectif.

9 Champ récepteur

Le champ récepteur d'un neurone est la région de l'image d'entrée susceptible d'influencer son activation.

Dans un CNN profond, le champ récepteur augmente avec le nombre de couches et les opérations de pooling.

Intuition : les premières couches voient un petit voisinage ; les couches profondes voient des régions plus larges, donc des motifs plus complexes.

10 Global Average Pooling

Si l'on a un tenseur de sortie

$$F \in \mathbb{R}^{H \times W \times C},$$

le global average pooling produit un vecteur $g \in \mathbb{R}^C$ défini par

$$g_k = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} F_{i,j,k}.$$

Avantages : très peu de paramètres, moins d'overfitting, lien naturel avec CAM et Grad-CAM.

11 Couches denses et classifieur final

Une couche dense calcule

$$z = Wx + b.$$

Si $x \in \mathbb{R}^n$ et qu'on veut m sorties, alors

$$W \in \mathbb{R}^{m \times n}, \qquad b \in \mathbb{R}^m.$$

Dans un réseau de classification, les dernières couches denses combinent des caractéristiques déjà extraites par les convolutions.

12 Softmax et interprétation probabiliste

Soient des logits $z_1, \ldots, z_K$. La fonction softmax est définie par

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}.$$

On a alors $p_i \in (0,1)$ et $\sum_i p_i = 1$.

13 Fonctions de perte

13.1 Cross-entropy binaire

Pour une cible $y \in \{0,1\}$ et une probabilité p, la perte est

$$L(y,p) = -\big(y \log p + (1-y) \log(1-p)\big).$$

13.2 Cross-entropy multi-classe

Si y est one-hot et p la sortie softmax :

$$L(y,p) = -\sum_{i=1}^{K} y_i \log p_i.$$

Si la vraie classe est c, cela se simplifie en

$$L = -\log p_c.$$

14 Descente de gradient

On cherche à minimiser une fonction de coût $J(\theta)$. La descente de gradient s'écrit

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t),$$

où $\eta > 0$ est le taux d'apprentissage.

14.1 Mini-batch SGD

En pratique, on n'utilise pas tout le dataset à chaque étape, mais de petits paquets (mini-batches) :

$$\theta_{t+1} = \theta_t - \eta \widehat{\nabla J}(\theta_t).$$

15 Backpropagation : principe général

La backpropagation est une application systématique de la règle de chaîne. Si

$$f(x) = f_3(f_2(f_1(x))),$$

alors

$$\frac{df}{dx} = \frac{df_3}{df_2}\frac{df_2}{df_1}\frac{df_1}{dx}.$$

16 Gradient d'une couche dense

Soit

$$z = Wx + b, \qquad L = L(z).$$

Alors

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial z} x^\top, \qquad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial z}, \qquad \frac{\partial L}{\partial x} = W^\top \frac{\partial L}{\partial z}.$$

17 Gradient d'une convolution : idée générale

Considérons une convolution simple

$$Y = X * W.$$

Alors le gradient par rapport au noyau W se calcule en corrélant l'entrée X avec le gradient de sortie, et le gradient par rapport à l'entrée X se calcule par une convolution du gradient de sortie avec le noyau renversé.

18 Architecture d'un CNN simple pour radiographies

Considérons l'architecture suivante, inspirée de ton projet :

1) entrée : $224 \times 224 \times 3$,
2) convolution 3×3, 16 filtres,
3) max pooling,
4) convolution 3×3, 32 filtres,
5) max pooling,
6) convolution 3×3, 64 filtres,
7) global average pooling,
8) dense 32,
9) dense 2.

18.1 Paramètres de la première couche

Chaque filtre a taille $3 \times 3 \times 3$, donc 27 poids, plus un biais. Avec 16 filtres :

$$(3 \times 3 \times 3 + 1) \times 16 = 448.$$

18.2 Deuxième couche convolutive

L'entrée a 16 canaux. Donc un filtre a taille $3 \times 3 \times 16$, soit 144 poids, plus un biais. Avec 32 filtres :

$$(144 + 1) \times 32 = 4640.$$

18.3 Troisième couche convolutive

L'entrée a 32 canaux. Donc

$$(3 \times 3 \times 32 + 1) \times 64 = 18496.$$

19 Régularisation

19.1 Weight decay

On peut ajouter un terme de pénalisation :

$$J_{\text{reg}}(\theta) = J(\theta) + \lambda \|\theta\|_2^2.$$

19.2 Dropout

Le dropout met aléatoirement certaines activations à zéro pendant l'entraînement.

19.3 Early stopping

On arrête l'entraînement quand la performance de validation cesse de s'améliorer.

20 Optimiseur Adam

Adam combine momentum et adaptation du pas d'apprentissage :

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2.$$

Puis on corrige les biais :

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}.$$

Enfin,

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

21 Mesures de performance

On définit TP, FP, TN, FN.

21.1 Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

21.2 Précision

$$\text{Precision} = \frac{TP}{TP + FP}.$$

21.3 Rappel

$$\text{Recall} = \frac{TP}{TP + FN}.$$

21.4 F1-score

$$F_1 = 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

22 Pourquoi l'explicabilité est nécessaire

En imagerie médicale, une bonne prédiction ne suffit pas. On veut savoir si le modèle regarde la bonne région anatomique. Sans cela, un modèle pourrait apprendre des artefacts du dataset.

23 Principe de Grad-CAM

Soit y^c le score associé à la classe cible c et A^k la k-ième feature map de la dernière couche convolutionnelle.

Grad-CAM définit un poids par canal :

$$\alpha_k^c = \frac{1}{Z}\sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k}.$$

La heatmap est ensuite :

$$L_{\text{GradCAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k\right).$$

24 Pourquoi la dernière couche convolutionnelle ?

Parce qu'elle conserve une structure spatiale tout en étant suffisamment sémantique. Les couches trop basses sont très locales et peu abstraites ; les couches denses ne conservent plus la géométrie spatiale.

25 Exemple jouet de Grad-CAM

Considérons trois feature maps :

$$A^1 = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \quad A^2 = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 2 & 2 \\ 1 & 2 & 1 \end{pmatrix}, \quad A^3 = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 8 & 2 \\ 1 & 2 & 1 \end{pmatrix}.$$

Supposons

$$\alpha_1^c = 0.1, \qquad \alpha_2^c = 0.3, \qquad \alpha_3^c = 0.9.$$

Alors

$$L^c = \text{ReLU}(0.1A^1 + 0.3A^2 + 0.9A^3).$$

La zone centrale sera la plus activée : cela signifie que le réseau justifie sa prédiction à partir de la région centrale.

26 Limites de Grad-CAM

Ce n'est pas une preuve causale, la heatmap dépend du choix de la couche, une heatmap visuellement jolie n'implique pas que le modèle soit correct, et il s'agit d'une explication locale, pas globale.

27 Exemple pratique en PyTorch

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.fc1 = nn.Linear(64, 32)
        self.fc2 = nn.Linear(32, 2)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = F.relu(self.conv3(x))
        x = x.mean(dim=(2, 3))
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

```
model = SmallCNN()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss()

for images, labels in train_loader:
    optimizer.zero_grad()
    logits = model(images)
    loss = criterion(logits, labels)
    loss.backward()
    optimizer.step()
```

28 Exemple pratique en TensorFlow / Keras

```
import tensorflow as tf
from tensorflow import keras

model = keras.Sequential([
    keras.layers.Input(shape=(224,224,3)),
    keras.layers.Conv2D(16, 3, activation='relu', padding='same'),
    keras.layers.MaxPool2D(),
```

```
    keras.layers.Conv2D(32, 3, activation='relu', padding='same'),
    keras.layers.MaxPool2D(),
    keras.layers.Conv2D(64, 3, activation='relu', padding='same', name='last_conv'),
    keras.layers.GlobalAveragePooling2D(),
    keras.layers.Dense(32, activation='relu'),
    keras.layers.Dense(2)
])
```

```
with tf.GradientTape() as tape:
    conv_outputs, predictions = grad_model(image_batch)
    class_channel = predictions[:, class_index]

grads = tape.gradient(class_channel, conv_outputs)
pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
heatmap = tf.reduce_sum(conv_outputs[0] * pooled_grads, axis=-1)
heatmap = tf.maximum(heatmap, 0)
heatmap = heatmap / tf.reduce_max(heatmap)
```

29 Lien avec le projet MedVision

Dans le projet MedVision : les images sont prétraitées puis injectées dans un CNN, un modèle baseline et un modèle optimisé peuvent être comparés, MLflow trace les expériences, FastAPI expose la prédiction, Streamlit permet une démonstration simple, et Grad-CAM explique la décision visuellement.

30 Exercices proposés

Exercice 1

Calculer à la main la convolution d'une matrice 4×4 avec un filtre 2×2.

Exercice 2

Montrer qu'une composition de fonctions affines reste affine. En déduire pourquoi les activations non linéaires sont indispensables.

Exercice 3

Déterminer le nombre de paramètres d'une couche convolutive de 32 filtres 5×5 appliquée à une entrée possédant 16 canaux.

Exercice 4

Comparer le nombre de paramètres entre une couche dense et une couche convolutive à partir d'une entrée $64 \times 64 \times 3$.

Exercice 5

Expliquer avec tes mots pourquoi Grad-CAM ne constitue pas une preuve de validité clinique.

31 Conclusion générale

Les réseaux convolutifs reposent sur des idées mathématiques relativement simples mais extrêmement puissantes : produits scalaires locaux, non-linéarités, descente de gradient et composition de fonctions. Leur efficacité vient du fait qu'ils exploitent la structure spatiale des images tout en apprenant automatiquement des représentations hiérarchiques.

L'explicabilité, avec des méthodes comme Grad-CAM, ne remplace pas l'évaluation scientifique d'un modèle, mais elle constitue un outil essentiel pour inspecter, critiquer et améliorer des systèmes d'IA appliqués à des données sensibles comme les images médicales.

A Bibliographie minimale conseillée

— I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning.*
— R. Szeliski, *Computer Vision : Algorithms and Applications.*
— Y. LeCun et al., travaux fondateurs sur les CNN.
— Selvaraju et al., *Grad-CAM : Visual Explanations from Deep Networks via Gradient-based Localization.*